

Общие условия

Выполнил

Радионов Никита Дмитриевич БПИ236.

Вариант задания

Вариант 4. Разработать программу, находящую в заданной ASCII-строке последнюю при перемещении слева направо последовательность из N символов, каждый элемент которой определяется по условию «больше предшествующего» (число N вводится как отдельный параметр). Реализация:

```
// Функция для нахождения последней возрастающей последовательности
void find_last_increasing_sequence(char *data, int N, char *result) {
    int length = strlen(data);
    int last_start = -1;

    // Ищем возрастающую последовательность
    for (int i = 0; i <= length - N; i++) {
        int is_increasing = 1;
        for (int j = i + 1; j < i + N; j++) {
            if (data[j] <= data[j - 1]) {
                is_increasing = 0;
                break;
            }
        }
        if (is_increasing) {
            last_start = i;
        }
    }

    if (last_start != -1) {
        strncpy(result, data + last_start, N);
        result[N] = '\0';
    } else {
        result[0] = '\0';
    }
}
```

При всех тестовых запусках значение параметра N равно 3.

В заданиях, требующих установки ограничения на размер буферов данных, в качестве размера буферов данных было выбрано значение, равное 3 Байт (3 ASCII-символа).

Входные файлы и ожидаемое содержимое выходных файлов

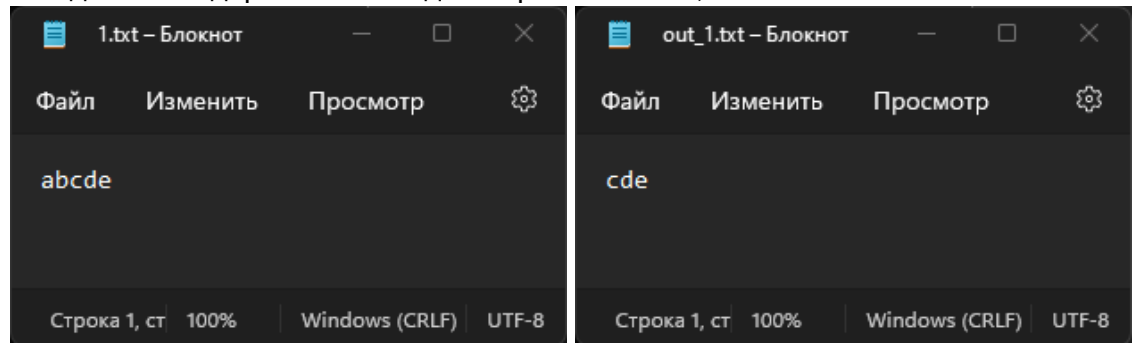
Всего подготовлены пять входных файлов для проверки работы разработанных программ на корректность:

- **1.txt**

Проверка на корректность обработки при стандартных условиях;

Содержимое файла: «abcde»;

Ожидаемое содержимое выходного файла: «cde»;

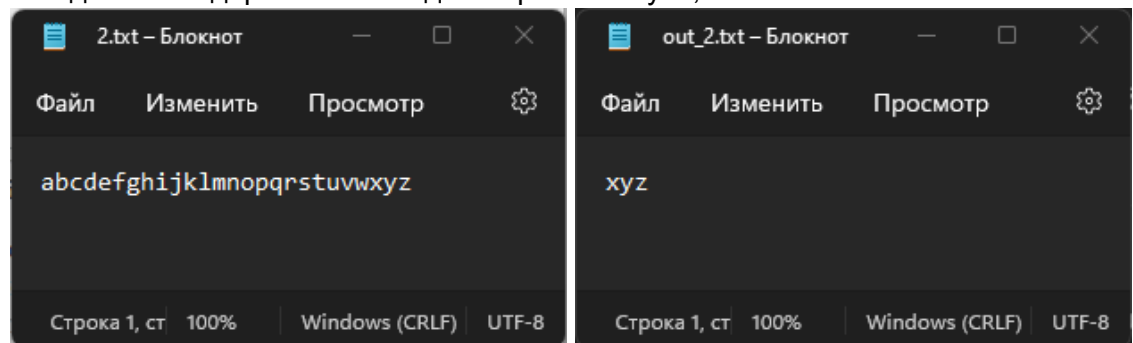


- **2.txt**

Проверка на корректность обработки при стандартных условиях;

Содержимое файла: «abcdefghijklmnopqrstuvwxyz»;

Ожидаемое содержимое выходного файла: «xyz»;

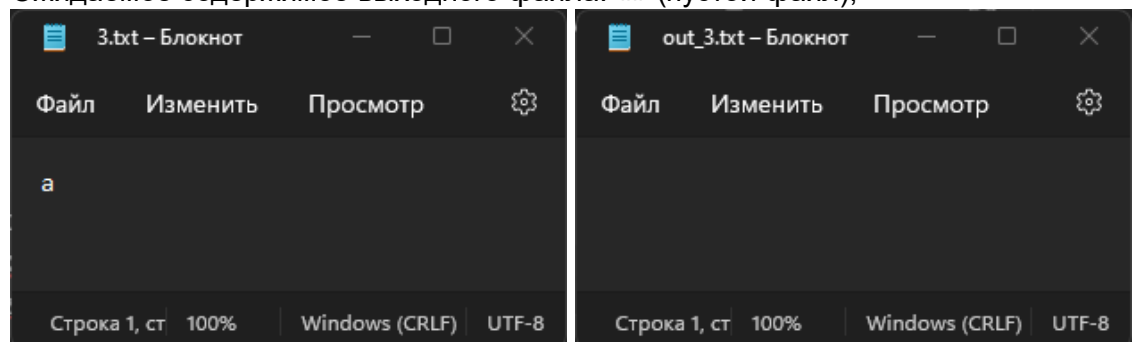


- **3.txt**

Проверка на корректность обработки при размере входного файла, меньшем чем значение параметра N;

Содержимое файла: «a»;

Ожидаемое содержимое выходного файла: «» (пустой файл);

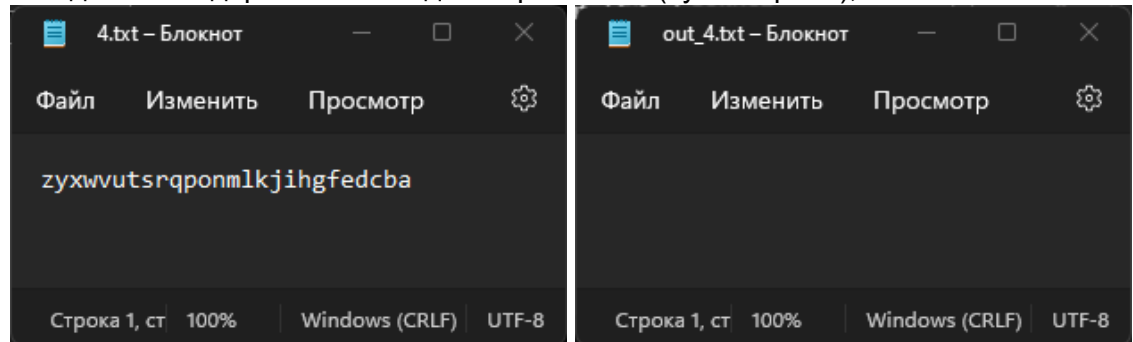


- **4.txt**

Проверка на корректность обработки при пустом входном файле;

Содержимое файла: «» (пустой файл);

Ожидаемое содержимое выходного файла: «» (пустой файл);

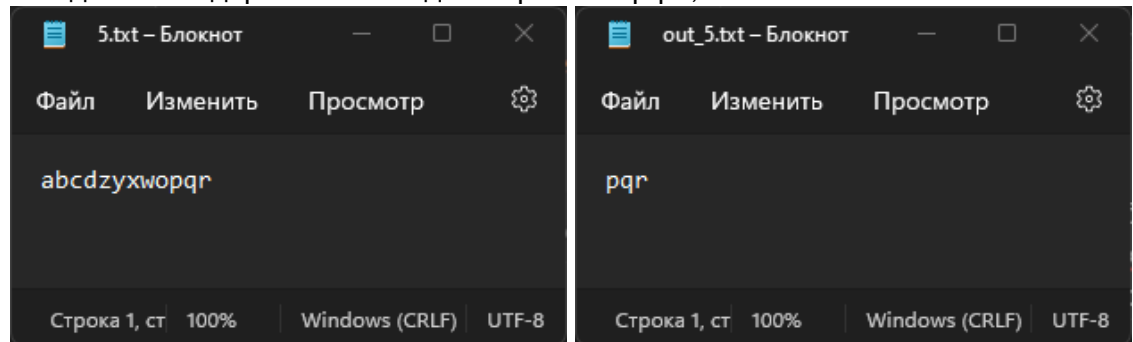


- **5.txt**

Проверка на корректность обработки при стандартных условиях;

Содержимое файла: «abcdzyxworqr»;

Ожидаемое содержимое выходного файла: «pqr»;

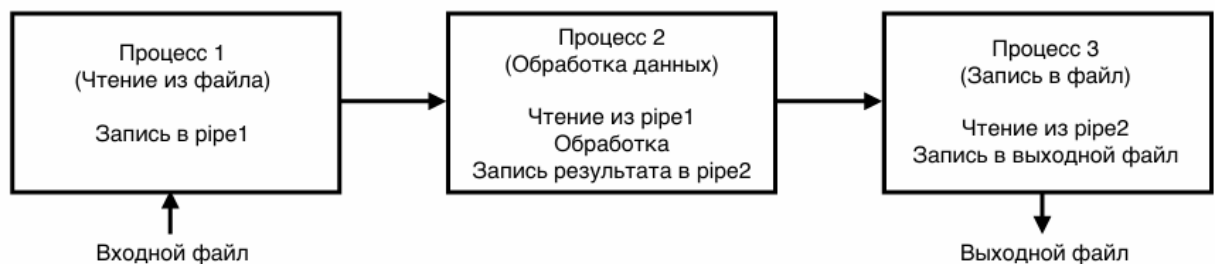


Задания

Задание на 4 балла

Разработка программы, осуществляющей взаимодействие между тремя родственными процессами одной программы с использованием неименованных каналов.

Общая схема решения



Выполнение требований

- В коде реализована проверка аргументов командной строки, где ожидаются три параметра: имя входного файла, имя выходного файла и значение N (размер последовательности). В случае неправильного числа аргументов программа выводит соответствующую ошибку:

```
int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Формат вызова: %s <input_file> <output_file> <N>\n", argv[0]);
        return 1;
    }

    char *input_file = argv[1];
    char *output_file = argv[2];
    int N = atoi(argv[3]);

    if (N <= 0) {
        fprintf(stderr, "N д.б больше 1!\n");
        return 1;
    }
}
```

- Реализация 3 процессов, взаимодействующих через неименованные каналы представлена в исходном коде:

```

// Процесс 1 - чтение из файла и передача через канал
void process_1(int pipe_fd, char *input_file) {
    int input_fd = open(input_file, O_RDONLY);
    if (input_fd == -1) {
        perror("Не открыть входной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read = read(input_fd, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        write(pipe_fd, buffer, bytes_read);
    }

    close(input_fd);
    close(pipe_fd);
}

// Процесс 2 - поиск последовательности и передача через канал
void process_2(int pipe_in, int pipe_out, int N) {
    char buffer[BUFFER_SIZE];
    char result[BUFFER_SIZE];
    ssize_t bytes_read = read(pipe_in, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        find_last_increasing_sequence(buffer, N, result);
        write(pipe_out, result, strlen(result));
    }

    close(pipe_in);
    close(pipe_out);
}

```

```

// Процесс 3 - запись в файл
void process_3(int pipe_fd, char *output_file) {
    int output_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd == -1) {
        perror("Не открыть выходной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read = read(pipe_fd, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        write(output_fd, buffer, bytes_read);
    }

    close(output_fd);
    close(pipe_fd);
}

```

```

int pipe1[2], pipe2[2];
if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
    perror("Ошибка при создании канала");
    return 1;
}

pid_t pid1 = fork();
if (pid1 == 0) {
    // Процесс 1
    close(pipe1[0]); // Закрыть ненужный конец
    process_1(pipe1[1], input_file);
    exit(0);
}

pid_t pid2 = fork();
if (pid2 == 0) {
    // Процесс 2
    close(pipe1[1]); // Закрыть ненужный конец
    close(pipe2[0]); // Закрыть ненужный конец
    process_2(pipe1[0], pipe2[1], N);
    exit(0);
}

pid_t pid3 = fork();
if (pid3 == 0) {
    // Процесс 3
    close(pipe2[1]); // Закрыть ненужный конец
    process_3(pipe2[0], output_file);
    exit(0);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

// Ожидание завершения всех процессов
wait(NULL);
wait(NULL);
wait(NULL);

```

- Для чтения данных из файла используется системный вызов read:

```
ssize_t bytes_read = read(pipe_in, buffer, BUFFER_SIZE);
```

- Для записи данных в файл используется системный вызов write:

```
write(output_fd, buffer, bytes_read);
```

- Буфер для хранения данных установлен размером 5000 байт:

```
#define BUFFER_SIZE 5000
```

- Размер входных файлов не превышает размер буфера (см. раздел «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета)

Результаты работы разработанной программы

Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

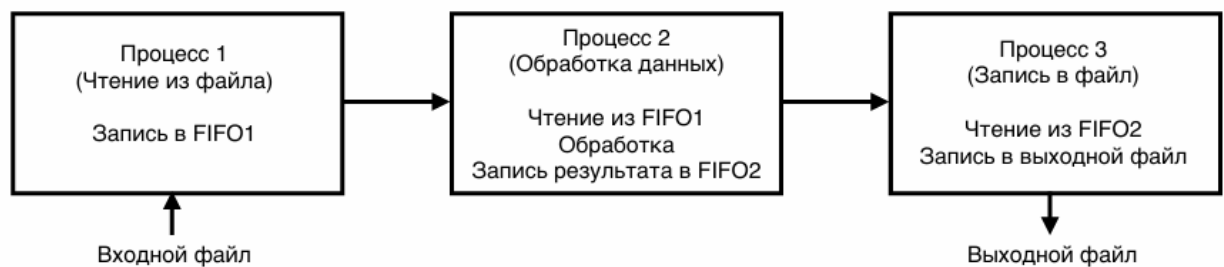
Результаты работы программы представлены в виде выходных файлов вида «out_<номер входного файла>.txt». Содержимое выходных файлов соответствует

ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 5 баллов

В дополнение к требованиям и программе на предыдущую оценку необходимо разработать еще одну программу, в которой взаимодействие между тремя родственными процессами одной программы осуществляется через именованные каналы.

Общая схема решения



Выполнение требований

Аналогично заданию на 4 балла, кроме использования именованных каналов:

```

// Указываем имена FIFO
const char *fifo1 = "../tmp/fifo1";
const char *fifo2 = "../tmp/fifo2";

// Создание именованных каналов (FIFO)
if (mkfifo(fifo1, 0666) == -1) {
    perror("Ошибка создания FIFO1");
    return 1;
}
if (mkfifo(fifo2, 0666) == -1) {
    perror("Ошибка создания FIFO2");
    return 1;
}

pid_t pid1 = fork();
if (pid1 == 0) {
    // Процесс 1
    int fifo1_fd = open(fifo1, O_WRONLY);
    if (fifo1_fd == -1) {
        perror("Не открыть FIFO1 для записи");
        exit(1);
    }
    process_1(fifo1_fd, input_file);
    exit(0);
}

pid_t pid2 = fork();
if (pid2 == 0) {
    // Процесс 2
    int fifo1_fd = open(fifo1, O_RDONLY);
    if (fifo1_fd == -1) {
        perror("Не открыть FIFO1 для чтения");
        exit(1);
    }

    int fifo2_fd = open(fifo2, O_WRONLY);
    if (fifo2_fd == -1) {
        perror("Не открыть FIFO2 для записи");
        exit(1);
    }

    process_2(fifo1_fd, fifo2_fd, N);
    exit(0);
}

```



```

pid_t pid3 = fork();
if (pid3 == 0) {
    // Процесс 3
    int fifo2_fd = open(fifo2, O_RDONLY);
    if (fifo2_fd == -1) {
        perror("Не открыть FIFO2 для чтения");
        exit(1);
    }
    process_3(fifo2_fd, output_file);
    exit(0);
}

// Ожидание завершения всех процессов
wait(NULL);
wait(NULL);
wait(NULL);

// Удаление FIFO после завершения
unlink(fifo1);
unlink(fifo2);

return 0;

```

Результаты работы разработанной программы

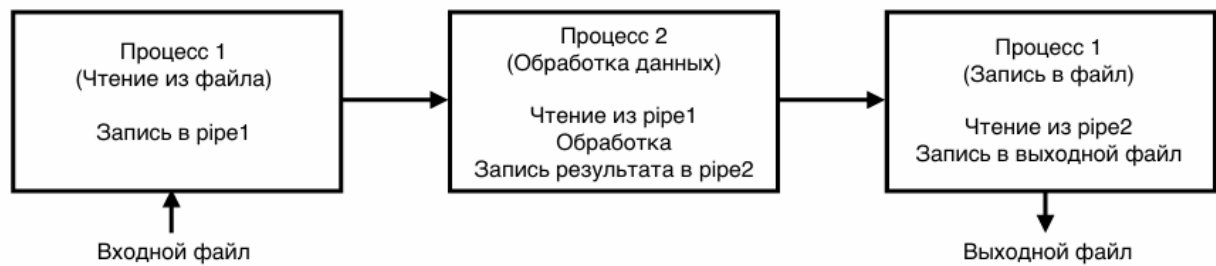
Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Результаты работы программы представлены в виде выходных файлов вида «*out_<номер входного файла>.txt*». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 6 баллов

В дополнение к требованиям на предыдущую оценку разработать новую программу, которая осуществляет взаимодействие между двумя родственными процессами одной программы с использованием неименованных каналов.

Общая схема решения



Выполнение требований

Аналогично предыдущим, но используются 2 процесса и неименованные каналы:

```
// Процесс 1 - чтение из файла и передача через канал
void process_1(int pipe_write_fd, char *input_file) {
    int input_fd = open(input_file, O_RDONLY);
    if (input_fd == -1) {
        perror("Не открыть входной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read = read(input_fd, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        write(pipe_write_fd, buffer, bytes_read);
    }

    close(input_fd);
    close(pipe_write_fd);
}

// Процесс 2 - поиск последовательности и передача через канал
void process_2(int pipe_read_fd, int pipe_write_fd, int N) {
    char buffer[BUFFER_SIZE];
    char result[BUFFER_SIZE];
    ssize_t bytes_read = read(pipe_read_fd, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        find_last_increasing_sequence(buffer, N, result);
        write(pipe_write_fd, result, strlen(result));
    }

    close(pipe_read_fd);
    close(pipe_write_fd);
}
```

```

// Создание неименованных каналов
int pipe1[2], pipe2[2];
if (pipe(pipe1) == -1) {
    perror("Ошибка при создании pipe1");
    return 1;
}
if (pipe(pipe2) == -1) {
    perror("Ошибка при создании pipe2");
    return 1;
}

pid_t pid1 = fork();
if (pid1 == 0) {
    // Процесс 1
    close(pipe1[0]); // Закрытие неиспользуемого конца
    close(pipe2[0]); // Закрытие неиспользуемого конца
    close(pipe2[1]); // Закрытие неиспользуемого конца
    process_1(pipe1[1], input_file); // Передача данных через канал
    exit(0);
}

pid_t pid2 = fork();
if (pid2 == 0) {
    // Процесс 2
    close(pipe1[1]); // Закрытие неиспользуемого конца
    close(pipe2[0]); // Закрытие неиспользуемого конца
    process_2(pipe1[0], pipe2[1], N); // Обработка данных и передача
    exit(0);
}

// Процесс 1 - после того, как процесс 2 завершит обработку
close(pipe1[0]); // Закрытие неиспользуемого конца
close(pipe1[1]); // Закрытие неиспользуемого конца
close(pipe2[1]); // Закрытие неиспользуемого конца

int output_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (output_fd == -1) {
    perror("Не открыть выходной файл");
    exit(1);
}

```

```
char buffer[BUFFER_SIZE];
ssize_t bytes_read = read(pipe2[0], buffer, BUFFER_SIZE);

if (bytes_read > 0) {
    write(output_fd, buffer, bytes_read);
}

close(output_fd);
close(pipe2[0]);

// Ожидание завершения всех процессов
wait(NULL);
wait(NULL);

return 0;
```

Результаты работы разработанной программы

Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Результаты работы программы представлены в виде выходных файлов вида «out_<номер входного файла>.txt». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 7 баллов

В дополнение к требованиям на предыдущую оценку разработать программу, которая осуществляет взаимодействие между двумя родственными процессами одной программы с использованием именованных каналов.

Общая схема решения



Выполнение требований

Аналогично предыдущему, за исключением использования именованных каналов:

```

// Указываем имена FIFO
const char *fifo1 = "../tmp/fifo1";
const char *fifo2 = "../tmp/fifo2";

// Создание именованных каналов (FIFO)
if (mkfifo(fifo1, 0666) == -1) {
    perror("Ошибка создания FIFO1");
    return 1;
}
if (mkfifo(fifo2, 0666) == -1) {
    perror("Ошибка создания FIFO2");
    return 1;
}

pid_t pid1 = fork();
if (pid1 == 0) {
    // Процесс 1
    int fifo1_fd = open(fifo1, O_WRONLY);
    if (fifo1_fd == -1) {
        perror("Не открыть FIFO1 для записи");
        exit(1);
    }
    process_1(fifo1_fd, input_file);
    exit(0);
}

pid_t pid2 = fork();
if (pid2 == 0) {
    // Процесс 2
    int fifo1_fd = open(fifo1, O_RDONLY);
    if (fifo1_fd == -1) {
        perror("Не открыть FIFO1 для чтения");
        exit(1);
    }

    int fifo2_fd = open(fifo2, O_WRONLY);
    if (fifo2_fd == -1) {
        perror("Не открыть FIFO2 для записи");
        exit(1);
    }

    process_2(fifo1_fd, fifo2_fd, N);
    exit(0);
}

```

```

// Процесс 1 - после того, как процесс 2 завершит обработку
int fifo2_fd = open(fifo2, O_RDONLY);
if (fifo2_fd == -1) {
    perror("Не открыть FIFO2 для чтения");
    exit(1);
}

int output_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (output_fd == -1) {
    perror("Не открыть выходной файл");
    exit(1);
}

char buffer[BUFFER_SIZE];
ssize_t bytes_read = read(fifo2_fd, buffer, BUFFER_SIZE);

if (bytes_read > 0) {
    write(output_fd, buffer, bytes_read);
}

close(output_fd);
close(fifo2_fd);

// Ожидание завершения всех процессов
wait(NULL);
wait(NULL);
wait(NULL);

// Удаление FIFO после завершения
unlink(fifo1);
unlink(fifo2);

return 0;

```

Результаты работы разработанной программы

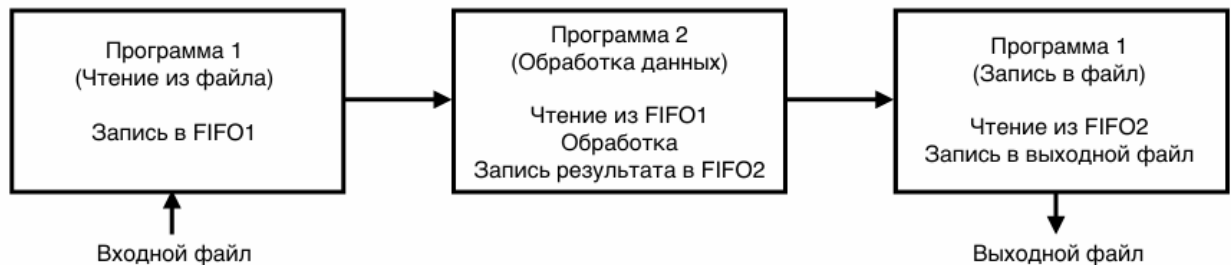
Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Результаты работы программы представлены в виде выходных файлов вида «out_<номер входного файла>.txt». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 8 баллов

В дополнение к требованиям на предыдущую оценку разработать программный продукт, в котором осуществляется взаимодействие между двумя независимыми процессами разных программ с использованием именованных каналов.

Общая схема решения



Выполнение требований

Сначала запускается первая программа, потом вторая. Формат вызова аналогичен предыдущим заданиям.

Аналогично предыдущим, но программа разбита на 2 независимые:

```

#define BUFFER_SIZE 5000

// ПРОГРАММА 1 - ЧТЕНИЕ И ПЕРЕДАЧА ДАННЫХ В ПРОЦЕСС 2 ЧЕРЕЗ FIFO

// Процесс 1 - чтение из файла и передача через канал
void process_1(int fifo_fd, char *input_file) {
    int input_fd = open(input_file, O_RDONLY);
    if (input_fd == -1) {
        perror("Не открыть входной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read = read(input_fd, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        write(fifo_fd, buffer, bytes_read);
    }

    close(input_fd);
    close(fifo_fd);
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Формат вызова: %s <input_file> <output_file> <N>\n", argv[0]);
        return 1;
    }

    char *input_file = argv[1];
    char *output_file = argv[2];
    int N = atoi(argv[3]);

    if (N <= 0) {
        fprintf(stderr, "N д.б больше 1!\n");
        return 1;
    }

    // Указываем имена FIFO
    const char *fifo1 = "../tmp/fifo1";
    const char *fifo2 = "../tmp/fifo2";

    // Создание именованного канала (FIFO)
    if (mkfifo(fifo1, 0666) == -1) {
        perror("Ошибка создания FIFO1");
        return 1;
    }
}

```



```
process_1(fifo1_fd, input_file); // Чтение из файла и передача через FIFO

// Ожидание завершения работы процесса 2
wait(NULL);

// Процесс 1 - после того, как процесс 2 завершит обработку
int fifo2_fd = open(fifo2, O_RDONLY);
if (fifo2_fd == -1) {
    perror("Не открыть FIFO2 для чтения");
    exit(1);
}

int output_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (output_fd == -1) {
    perror("Не открыть выходной файл");
    exit(1);
}

char buffer[BUFFER_SIZE];
ssize_t bytes_read = read(fifo2_fd, buffer, BUFFER_SIZE);

if (bytes_read > 0) {
    write(output_fd, buffer, bytes_read);
}

close(output_fd);
close(fifo2_fd);

// Удаление FIFO после завершения
unlink(fifo1);
unlink(fifo2);

return 0;
```

```

// ПРОГРАММА 2 - ОБРАБОТКА И ПЕРЕДАЧА ДАННЫХ В ПРОЦЕСС 1 ЧЕРЕЗ FIFO

// Функция для нахождения последней последовательности из N символов, где каждый больше предыдущего
void find_last_increasing_sequence(char *data, int N, char *result) {
    int length = strlen(data);
    int last_start = -1;

    // Ищем последовательности, где каждый символ больше предыдущего
    for (int i = 0; i <= length - N; i++) {
        int is_increasing = 1;
        for (int j = i + 1; j < i + N; j++) {
            if (data[j] <= data[j - 1]) {
                is_increasing = 0;
                break;
            }
        }
        if (is_increasing) {
            last_start = i; // запоминаем последний индекс начала последовательности
        }
    }

    if (last_start != -1) {
        strncpy(result, data + last_start, N);
        result[N] = '\0'; // заканчиваем строку
    } else {
        result[0] = '\0'; // если последовательность не найдена
    }
}

// Процесс 2 - поиск последовательности и передача через именованный канал
void process_2(int fifo_in, int fifo_out, int N) {
    char buffer[BUFFER_SIZE];
    char result[BUFFER_SIZE];
    ssize_t bytes_read = read(fifo_in, buffer, BUFFER_SIZE);

    if (bytes_read > 0) {
        find_last_increasing_sequence(buffer, N, result);
        write(fifo_out, result, strlen(result));
    }

    close(fifo_in);
    close(fifo_out);
}

```

```

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Формат вызова: %s <input_file> <output_file> <N>\n", argv[0]);
        return 1;
    }

    char *input_file = argv[1];
    char *output_file = argv[2];
    int N = atoi(argv[3]);

    if (N <= 0) {
        fprintf(stderr, "N д.б больше 1!\n");
        return 1;
    }

    // Указываем имена FIFO
    const char *fifo1 = "../tmp/fifo1";
    const char *fifo2 = "../tmp/fifo2";

    // Создание именованного канала (FIFO)
    if (mkfifo(fifo2, 0666) == -1) {
        perror("Ошибка создания FIFO2");
        return 1;
    }

    int fifo1_fd = open(fifo1, O_RDONLY);
    if (fifo1_fd == -1) {
        perror("Не открыть FIFO1 для чтения");
        exit(1);
    }

    int fifo2_fd = open(fifo2, O_WRONLY);
    if (fifo2_fd == -1) {
        perror("Не открыть FIFO2 для записи");
        exit(1);
    }

    process_2(fifo1_fd, fifo2_fd, N); // Обработка данных и передача результата

    // Удаление FIFO после завершения
    unlink(fifo2);

    return 0;
}

```

Результаты работы разработанной программы

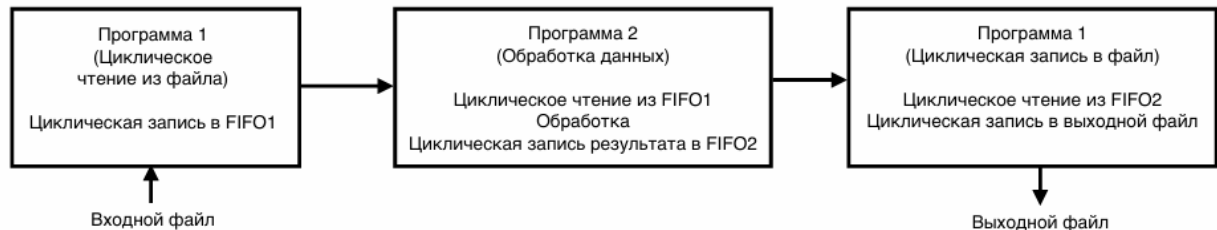
Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Результаты работы программы представлены в виде выходных файлов вида «*out_номер входного файла.txt*». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 9 баллов

В дополнение к требованиям на предыдущую оценку разработать программный продукт, в котором осуществляется взаимодействие между двумя независимыми процессами разных программ с использованием именованных каналов. Обмен данными ведется за счет многократной передачи текста через ограниченные по размеру буферы.

Общая схема решения



Выполнение требований

Сначала запускается первая программа, потом вторая. Формат вызова аналогичен предыдущим заданиям.

Аналогично предыдущему, но так как размер буфера ограничен, чтение и запись происходят в цикле. Кроме того, размер входных файлов с номерами 2 и 5 превышает размер буфера:

```
#define BUFFER_SIZE 5
```

```
// ПРОГРАММА 1 - ЧТЕНИЕ И ПЕРЕДАЧА ДАННЫХ В ПРОЦЕСС 2 ЧЕРЕЗ FIFO

// Процесс 1 - чтение из файла и передача через канал
void process_1(int fifo_fd, char *input_file) {
    int input_fd = open(input_file, O_RDONLY);
    if (input_fd == -1) {
        perror("Не открыть входной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(input_fd, buffer, BUFFER_SIZE)) > 0) {
        // Передаем данные через FIFO в цикле
        write(fifo_fd, buffer, bytes_read);
    }

    close(input_fd);
    close(fifo_fd);
}
```

Во второй программе:

```
// Процесс 2 - поиск последовательности в цикле и передача через именованный канал в цикле
void process_2(int fifo_in, int fifo_out, int N) {
    char buffer[BUFFER_SIZE];
    char result[BUFFER_SIZE];
    char *full_data = NULL; // Динамически выделяемый буфер для всей строки
    int data_index = 0;
    ssize_t bytes_read;
    size_t buffer_size = 0;

    // Читаем все данные из FIFO и собираем их в динамический буфер
    while ((bytes_read = read(fifo_in, buffer, BUFFER_SIZE)) > 0) {
        // Перераспределяем память для хранения новых данных
        buffer_size += bytes_read;
        full_data = realloc(full_data, buffer_size + 1); // +1 для нулевого символа

        if (full_data == NULL) {
            perror("Ошибка при выделении памяти");
            exit(1);
        }

        memcpy(full_data + data_index, buffer, bytes_read);
        data_index += bytes_read;
    }

    // Завершаем строку
    full_data[data_index] = '\0';

    // Ищем последнюю возрастающую последовательность по всей строке
    find_last_increasing_sequence(full_data, N, result);
    write(fifo_out, result, strlen(result)); // Отправляем результат

    // Освобождаем память
    free(full_data);

    close(fifo_in);
    close(fifo_out);
}
```

Результаты работы разработанной программы

Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

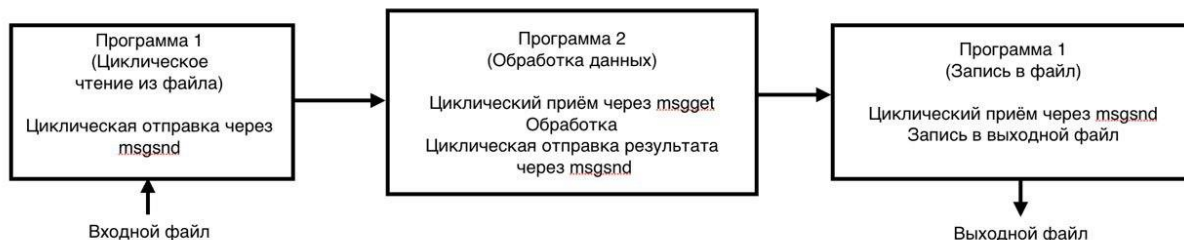
Результаты работы программы представлены в виде выходных файлов вида «*out_<номер входного файла>.txt*». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Задание на 10 баллов

В дополнение к требованиям на предыдущую оценку разработать программный продукт, осуществляющий взаимодействие между двумя независимыми процессами разных программ с использованием очередей сообщений. Обмен данными ведется за счет

многократной передачи текста через ограниченные по размеру буферы. Тип очередей сообщений можно выбирать по желанию.

Общая схема решения



Выполнение требований

Запускается только первая программа.

Аналогично предыдущим, но используются очереди сообщений, и вторая программа вызывается из первой посредством вызова `exescl`:

Программа 1:

```
#define BUFFER_SIZE 5
#define MSG_KEY1 1234 // Ключ для очереди сообщений для передачи данных
#define MSG_KEY2 5678 // Ключ для очереди сообщений для получения результата

struct message {
    long msg_type;
    char data[BUFFER_SIZE];
};

void process_1(int msgid_send, char *input_file) {
    int input_fd = open(input_file, O_RDONLY);
    if (input_fd == -1) {
        perror("Не открыть входной файл");
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    // Читаем файл и отправляем данные порциями
    while ((bytes_read = read(input_fd, buffer, BUFFER_SIZE)) > 0) {
        struct message msg;
        msg.msg_type = 1; // Тип сообщения для второго процесса
        memcpy(msg.data, buffer, bytes_read);
        if (msgsnd(msgid_send, &msg, bytes_read, 0) == -1) {
            perror("Ошибка отправки сообщения");
            exit(1);
        }
        printf("Отправлено в очередь: %.*s\n", (int)bytes_read, msg.data); // Отладочный вывод
    }

    close(input_fd);
}
```

```

void write_to_file(char *output_file, char *data, ssize_t bytes) {
    int output_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd == -1) {
        perror("Не открыть выходной файл");
        exit(1);
    }
    write(output_fd, data, bytes);
    close(output_fd);
    printf("Записано в файл: %.*s\n", (int)bytes, data); // Отладочный вывод
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Формат вызова: %s <input_file> <output_file> <N>\n", argv[0]);
        return 1;
    }

    char *input_file = argv[1];
    char *output_file = argv[2];
    int N = atoi(argv[3]);

    if (N <= 0) {
        fprintf(stderr, "N должно быть больше 1!\n");
        return 1;
    }

    // Получаем доступ к очередям сообщений
    int msgid_send = msgget(MSG_KEY1, 0666 | IPC_CREAT);
    if (msgid_send == -1) {
        perror("Не удалось создать очередь сообщений для отправки");
        exit(1);
    }

    int msgid_receive = msgget(MSG_KEY2, 0666 | IPC_CREAT);
    if (msgid_receive == -1) {
        perror("Не удалось создать очередь сообщений для получения");
        exit(1);
    }

    // Процесс 1 - Чтение из файла и передача через очередь сообщений
    process_1(msgid_send, input_file);

    // Запускаем программу 2 для обработки данных

```

```

// Запускаем программу 2 для обработки данных
pid_t pid = fork();
if (pid == 0) {
    // Дочерний процесс (процесс 2) вызывает программу 2
    execl("./executable2", "./executable2", input_file, output_file, argv[3], NULL);
    perror("Ошибка при вызове execl");
    exit(1);
} else if (pid > 0) {
    // Родительский процесс получает обработанные данные и записывает в файл
    struct message msg;
    ssize_t bytes_received;
    while ((bytes_received = msgrcv(msgid_receive, &msg, BUFFER_SIZE, 0, 0)) > 0) {
        printf("Получено сообщение: %.*s\n", (int)bytes_received, msg.data); // Отладочный вывод
        write_to_file(output_file, msg.data, bytes_received);
    }

    // Ожидаем завершения дочернего процесса
    waitpid(pid, NULL, 0); // Дожидаемся завершения дочернего процесса

    // Удаление очередей сообщений
    msgctl(msgid_send, IPC_RMID, NULL);
    msgctl(msgid_receive, IPC_RMID, NULL);
} else {
    perror("Ошибка при вызове fork");
    exit(1);
}

return 0;

```

Программа 2:


```

#define BUFFER_SIZE 5
#define MSG_KEY1 1234 // Ключ для очереди сообщений для получения данных
#define MSG_KEY2 5678 // Ключ для очереди сообщений для отправки результата

struct message {
    long msg_type;
    char data[BUFFER_SIZE];
};

// Функция для нахождения последней возрастающей последовательности
void find_last_increasing_sequence(char *data, int N, char *result) {
    int length = strlen(data);
    int last_start = -1;

    // Ищем возрастающую последовательность
    for (int i = 0; i <= length - N; i++) {
        int is_increasing = 1;
        for (int j = i + 1; j < i + N; j++) {
            if (data[j] <= data[j - 1]) {
                is_increasing = 0;
                break;
            }
        }
        if (is_increasing) {
            last_start = i;
        }
    }

    if (last_start != -1) {
        strncpy(result, data + last_start, N);
        result[N] = '\0';
    } else {
        result[0] = '\0';
    }
}

// Процесс 2 - обработка данных и передача через очередь сообщений
void process_2(int msgid_receive, int msgid_send, int N) {
    struct message msg;
    char full_data[BUFFER_SIZE * 10]; // Буфер для накопления данных
    int data_index = 0;
    ssize_t bytes_received;
    int end_of_data = 0; // Флаг для окончания данных

    // Читаем все сообщения из очереди

```

```

// Читаем все сообщения из очереди
while (!end_of_data) {
    bytes_received = msgrcv(msgid_receive, &msg, BUFFER_SIZE, 0, IPC_NOWAIT); // Используем IPC_NOWAIT

    if (bytes_received > 0) {
        printf("Получено сообщение в процессе 2: %.*s\n", (int)bytes_received, msg.data); // Отладочный вывод
        memcpy(full_data + data_index, msg.data, bytes_received);
        data_index += bytes_received;
    } else if (bytes_received == -1) {
        // Если очередь пуста, завершаем обработку
        break;
    }
}

// Завершаем строку
full_data[data_index] = '\0';

// Обрабатываем данные
char result[BUFFER_SIZE];
find_last_increasing_sequence(full_data, N, result);

// Отправляем результат обратно
msg.msg_type = 1;
memcpy(msg.data, result, strlen(result) + 1);
if (msgsnd(msgid_send, &msg, strlen(result) + 1, 0) == -1) {
    perror("Ошибка отправки сообщения");
    exit(1);
}
printf("Отправлено обратно в очередь: %s\n", result); // Отладочный вывод
}

```

```

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Формат вызова: %s <input_file> <output_file> <N>\n", argv[0]);
        return 1;
    }

    int N = atoi(argv[3]);
    if (N <= 0) {
        fprintf(stderr, "N должно быть больше 1!\n");
        return 1;
    }

    // Получаем доступ к очередям сообщений
    int msgid_receive = msgget(MSG_KEY1, 0666 | IPC_CREAT);
    if (msgid_receive == -1) {
        perror("Не удалось создать очередь сообщений для получения");
        exit(1);
    }

    int msgid_send = msgget(MSG_KEY2, 0666 | IPC_CREAT);
    if (msgid_send == -1) {
        perror("Не удалось создать очередь сообщений для отправки");
        exit(1);
    }

    // Процесс 2 - Обработка данных
    process_2(msgid_receive, msgid_send, N);

    // Удаляем очереди сообщений после завершения
    msgctl(msgid_receive, IPC_RMID, NULL);
    msgctl(msgid_send, IPC_RMID, NULL);

    return 0;
}

```

Результаты работы разработанной программы

Входные файлы для проведения тестовых запусков соответствуют файлам, представленным в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.

Результаты работы программы представлены в виде выходных файлов вида «*out_номер входного файла.txt*». Содержимое выходных файлов соответствует ожидаемому содержимому соответствующих файлов, представленному в разделе «Входные файлы и ожидаемое содержимое выходных файлов» настоящего отчета.